Wireless Personal Communications (2025) 140:739–775
https://doi.org/10.1007/s11277-025-11748-w

RESEARCH

# Assessing SDN Controller Vulnerabilities: A Survey on Attack Typologies, Detection Mechanisms, Controller Selection, and Dataset Application in Machine Learning

**Juliana Arevalo-Herrera[1,3] · Jorge Camargo Mendoza[2] · Jose Ignacio Martínez Torre[1] · Tatiana Zona-Ortiz[3] · Juan M. Ramirez[4]**

Accepted: 6 February 2025 / Published online: 27 February 2025


**Abstract**
SDN controllers become the main advantage of the architecture because they present a centralized control decision-making and general view of the network. They are, however, also a critical point that an attacker could exploit. More review of the body of research is needed regarding the types of attacks on SDN controllers, methods to detect them, and mitigation techniques directed specifically to the controller, particularly considering the approach of machine learning detection methods. This survey addresses the topics of attacks targeting the SDN controller, methods for their detection, what types of controllers are used in different studies, and datasets used in machine learning detection methods. The findings highlight that most attacks exploit vulnerabilities inherent in the OpenFlow protocol, while the detection methodologies remain primarily statistical and machine learning approaches. Additionally, the review shows that while outdated controllers like Floodlight and Ryu are still widely used in studies, actively supported controllers such as ONOS and ODL are used much less. Finally, the survey finds only two publicly available datasets tailored for SDN environments, none considering attacks directed at the controllers, illustrating a notable gap in the existing research. This survey also highlights the need for further research focusing on modern SDN controllers and developing comprehensive datasets to advance effective security solutions.

**Keywords** SDN · 5G · Machine learning · Attacks · Detection

## 1 Introduction

Software Defined Networks (SDNs) represent a paradigm for separating the control and data planes, allowing a centralized entity to act as the network operating system. In a traditional network, both planes are integrated into each device. Each network operation is either completely distributed (e.g., routing, switching) or performed in a middlebox with

Jorge Camargo Mendoza, Jose Ignacio Martínez Torre, Angela Tatiana Zona-Ortiz, and Juan Marcos Ramirez contributed equally to this work.

Extended author information available on the last page of the article

the resulting bottleneck (e.g., load balancing, IPS, IDS). In contrast, SDN considers dedicated forwarding devices (data plane) that are only responsible for forwarding according to a list of rules, while the decision-making is done in a separate instance called the controller (controller plane) that defines the rules according to the traffic and the specific network application. This architecture provides advantages like the possibility of virtualization of the network, creation of slices to separate different operations, new management, and security methods thanks to the centralized control, among others [1]. The OpenFlow (OF) protocol, proposed in 2008 [2], played a key role in enabling SDN by defining the interface between the controller and the data plane. While the protocol has evolved through five versions, other technologies such as P4 [3], which allows the data plane to be programmed, have also contributed to the advancement of SDN. Nevertheless, OpenFlow remains the de facto standard for connecting controllers and switches in industrial and academic contexts [1]. SDN has been a fascinating area of research for the past decade, and its relevance has increased recently with the integration of SDN and Network Function Virtualization (NFV) to address the need for programmable control and management in 5G/6G networks [4]. SDN architecture presents a different set of challenges compared to traditional networks, especially regarding security. In this sense, further research is needed to improve the security of SDN architectures, especially in the context of OpenFlow-based SDNs [5]. The centralization of control in SDNs, which can create bottlenecks and single points of failure, poses potential security risks that must be addressed.

SDN security research has been conducted from two perspectives: SDN is an enabler of innovative methods to improve the security of network resources and the intrinsic security of SDN itself. However, there remains a gap in understanding how machine learning can identify attacks targeted at the controller in an OpenFlow-based SDN. More research is needed on the types of attacks on SDN controllers, methods to detect them, and mitigation techniques specifically targeting the controller, especially considering the approach of machine learning detection methods.

Although other authors have provided a vision of the SDN security landscape [6–8], there is insufficient attention on the control plane and the OpenFlow protocol acting as the southbound interface. The study by Bhuiyan et al. [9] presents a specific approach to the control plane and provides an interesting insight: to include intrinsic security mechanisms in the controller to maintain the overall security of SDN architectures. However, it does not present a perspective on how to use machine learning to implement detection and defense. Authors in [10] present a comprehensive review of SDN security. In particular, the analysis of the control plane presents ten types of threats, although it does not present the detection and mitigation techniques. The survey presented by Singh et al. [11] considers a broad perspective in SDN architecture and analyzes the complete architecture regarding DoS/DDoS attacks. Considering the control plane, it presents a list of 3 security challenges: Application Threats, Scalability Threats, and DoS/DDoS Attacks. The survey also identifies seven types of DDoS attacks for SDN and classifies only two of them (packet-in flooding and controller saturation) in the control plane. Several studies point to the need to characterize the southbound interface. In particular, Han et al. [12] present an overview of the attacks directed at the controller with a classification into three groups: DoS/DDoS, spoofing, and malicious injection. Although it is an important contribution, it is over five years old, and several studies have presented new attacks and countermeasures for the SDN control plane. The presentation is of the mitigation proposal rather than the attack itself, which is different from this paper. As presented in Table 1, previous surveys have failed to address several critical aspects. They lack perspective on controller implementation types, omit reviews of datasets for machine learning methods,

**Table 1** Existing SDN security related surveys

| Ref | Year | Target | Contribution | Limitation |
|---|---|---|---|---|
| [12] | 2019 | Security threats, vulnerabilities and issues at the control plane | Classification of the attacks into DoS/DDoS, Spoofing and Malicious injection/Anomaly attacks | Lacks situation from the perspective of the type of controller implementation |
| [11] | 2020 | DDoS attacks in SDN architecture | DDoS attack type identification and classification of defense mechanism into four categories: Information theory, Machine learning, Artificial neural networks, and Others | It is focused solely on DDoS attacks and lacks a review of prominent datasets for machine learning methods |
| [10] | 2022 | Secure communication infrastructure provided by SDN | Taxonomy of security threats, detection and mitigation techniques in SDN for each layer, along with use cases | Lacks perspective of machine learning detection or defense implementation techniques |
| [6] | 2023 | SDN threats and mitigations in the complete architecture | Identification of 9 types of threat in the network along with 7 types of attacks and the aspect of security affected by it (availability, confidentiality, integrity). Compound of the main mitigation techniques in all layers | Lacks perspective of machine learning detection or defense implementation techniques |
| [7] | 2021 | DDoS attacks in SDN networks due to packet-in only | DoS and DDoS attacks in SDNs through the lenses of intrinsic and extrinsic approaches | It is focused solely on DDoS attacks and lacks a review of prominent datasets for machine learning methods |
| [9] | 2023 | Analysis of security in SDN control plane | Classification of the various attack vectors and countermeasures of SDN attacks from the perspective of the control layer | Lacks perspective of machine learning detection or defense implementation techniques |

and inadequately cover machine learning detection and defense techniques. These gaps limit our understanding of security challenges and solutions in the field.

The purpose of this study is to review the recent literature on attacks on the control plane of the SDN architecture, with special attention to Openflow networks, since it is the mainstream implementation. Providing this study is essential to implement secure SDN networks in emerging technologies such as IoT or 5G/6G mobile networks. This study's main contribution is the identification of SDN attacks directed specifically to the controller and Openflow protocol, along with recent machine learning detection methods and mitigation techniques. This paper aims to address this topic by examining the issue through the following four research questions:

1. What are the primary types of attacks targeting the SDN OpenFlow controller?
2. What methods or strategies can effectively detect attacks on the SDN controller?
3. Which specific SDN controller is utilized in the given study?
4. What suitable datasets can be used for training a machine learning model to detect attacks directed to the controller?

The literature review addresses the issue of attacks targeting the SDN controller. The specific attacks of interest include packet-in attacks, saturation plane attacks, and topology poisoning attacks. The objective was to analyze existing research and identify relevant studies that addressed these attack vectors. The review primarily included studies published since 2019, with the earliest inclusion date being 2015 to capture foundational work. Multiple databases were used to ensure comprehensive coverage, including Springerlink, Sciencedirect, IEEE Xplore, Taylor and Francis, and ACM.

The review process involved conducting systematic searches using appropriate keywords and Boolean operators to retrieve relevant articles. The search terms included variations of the attack names and SDN controller-related concepts. The initial search results were screened based on their titles and abstracts to assess their relevance to the research topic. The selected articles underwent a thorough reading of their full text to extract pertinent information regarding attack characteristics, detection methods, controllers, collection methods (if any), and datasets used for analysis. The references of the selected articles were also examined to identify additional relevant sources that may have been missed in the initial search. Through this methodology, the literature review aimed to provide a comprehensive overview of the existing knowledge on attacks targeting the SDN controller, without specifically focusing on machine learning algorithms to avoid the large number of papers that only use existing datasets for algorithm application.

The rest of the paper is organized as follows: Sect. 2 presents a background with the main concepts used in this review; Sect. 3 describes the primary types of attacks directed to the controller; Sect. 4 reviews attack detection methods; Sect. 5 surveys defense actions found in the literature; Sect. 6 presents the most common controllers used in the reviewed papers; Sect. 7 describes the limited number of datasets commonly used for anomaly detection in network traffic; Sect. 8 discusses the findings and a prospective; finally, Sect. 10 concludes the paper.

## 2 Background

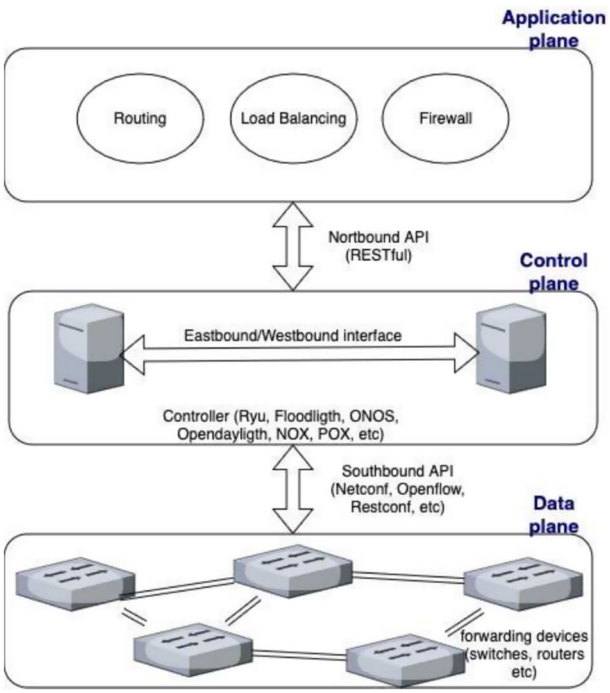
### 2.1 Software Defined Networks

SDN has emerged as a solution to address the inflexible integration of network equipment. By decoupling the control plane from the data plane, facilitated by the OpenFlow protocol [2], SDN enables the definition of network functions such as routing, firewall, load balancing, and bandwidth optimization as software applications running on top of the control plane. The SDN architecture is divided into three components: the data plane, consisting of switches; the control plane, featuring one or more controllers; and the application plane, which houses one or more network applications. Figure 1 presents the layers and interfaces of the architecture.

### 2.2 Openflow

The control plane is incorporated within network devices in traditional networking, resulting in entirely distributed control. This configuration makes dynamic and flexible network management and control challenging to implement. OpenFlow pioneered the concept of separating the control and data planes. Although earlier approaches existed, the proposal in McKeown et al. [2] ignited momentum for SDN. The Open Networking Foundation has since released five versions (1.0 to 1.5), each introducing enhancements and improvements to the protocol, such as increased scalability, support for IPv6, Quality of Service (QoS) features, and improved security mechanisms. The OpenFlow protocol provides a set of specifications that serve as the southbound interface, facilitating communication between

**Fig. 1** SDN architecture



the controller and forwarding devices to control their behavior and gather statistics. OpenFlow operation is defined by a set of messages including packet forwarding, packet dropping, statistical management, and flow table management.

The OpenFlow 1.5 specification considers using three type of messages with size limited to 64KB. The messages fall into three categories: Controller-to-Switch Messages, Asynchronous Messages, and Symmetric Messages. The first category includes Handshake, Switch Configuration, and Flow Table Configuration, among others. The second category comprises the Packet-in Message, Flow Removed Message, and the Port Status Message. The third category includes messages like Hello, Echo Request and Echo Reply, and Error Message. Figure 2 presents OpenFlow's basic operation, which relies on three key messages: Packet_in, Flow_mod, and Port_status. These messages enable the definition of flow tables in switches and provide network information to the controller. However, OpenFlow includes over fifty message types, and implementations offer flexibility in specifying operations. For example, the controller can control the type of asynchronous messages it receives, limiting switch communication methods. Moreover, OpenFlow specifications support message bundling, allowing multiple requests to be executed as a single unit. These bundled messages are processed simultaneously and are either all implemented or rejected if any errors are detected. This capability, along with other features, enhances OpenFlow's effectiveness as a robust control protocol for SDN.

One of the initial procedures an OpenFlow controller performs is activating the OpenFlow Discovery Protocol triggered by an OFPT_PACKET_OUT message, based on the Link Layer Discovery Protocol, aiming to identify nodes and links in the network. This protocol enables the controller to obtain a comprehensive topology view and update this information accordingly.

Conversely, when an OpenFlow switch receives a packet, it checks against the flow table, an internal data structure within the switch that dictates how to handle the packet based on features such as TCP/UDP port, IP address, MAC address, or in-port. If there is no flow table entry corresponding to a specific packet, a "table-miss" occurs. OpenFlow mandates that the switch sends an asynchronous "packet-in" message to the controller in such instances. Subsequently, the controller will determine the optimal handling method for the flow and dispatch a "flow-mod" message to create a new entry in

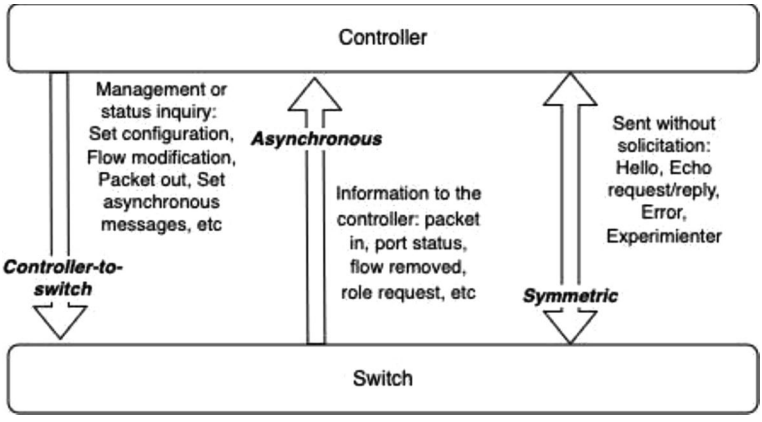



**Fig. 2** Openflow messages

the flow-table. This procedure is the most direct method to connect the controller from the data plane and is frequently exploited by attackers.

The elements above constitute the main vulnerabilities exploited in SDN to attack the controller, which are detailed in the subsequent sections.

## 3 Attacks Directed to the Controller

This section addresses the first research question concerning the primary types of attacks targeting the SDN controller. Here, we postulate that these attacks aim to disrupt the normal functioning of the OpenFlow controller, even if the direct target of the attack might be a network element or the control channel itself.

Within the window of literature search, a total of studies presented attacks and defenses directed exclusively to controllers. The main categories of attacks are:

- Denial of Service: The objective is to incapacitate the controller through overload, typically by flooding it with packet-in messages.
- Topology Poisoning: This attack involves feeding the controller false topology information to disrupt its overall network view.
- Bandwidth Overload: The objective entails flooding traffic to overload network links.

However,we identified that several studies focused on different types of attacks, including Man in the Middle (MITM) attacks [13] and Advanced Persistent Threat (APT) attacks [14]. Figure 3 presents a taxonomy of the attacks.

In the subsequent sections, we will delve into these distinct attack categories. Notably, many studies primarily concentrate on presenting defense strategies and detection mechanisms rather than explicitly describing the attacks themselves.

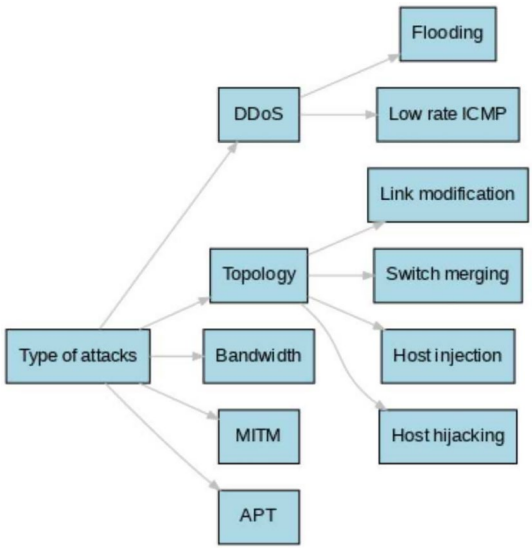



**Fig. 3** Types of attacks found in the survey directed to SDN controller

### 3.1 Denial of Service Attack

Various authors have investigated mechanisms for Denial of Service (DoS) attacks targeting the controller, including packet-in injection through IP and MAC spoofing, TCP SYN attacks, and saturation attacks, with a particular focus on the Ryu controller. In this section, we review the existing DoS studies, taking into account the specific controller software utilized in each case.

It is noteworthy that the studies investigate denial-of-service attacks targeting the controller. One common characteristic is that packet-in messages are utilized as the means of attack. This method represents the predominant approach for gaining access to the controller from the data plane.

Only one of the papers examined in this section does not employ the packet-in flooding as an attack method. This exception is the study in [15], which uses a low-rate flow that does not spoof any address. Instead, it exploits a vulnerability in the Internet Control Messaging Protocol (ICMP) and leverages many hosts as bots.

The first study of interest is presented by Chen et al. [16], which presents a comprehensive proposal for detecting and mitigating DDoS attacks aimed at the controller. However, the study does not specify the controller used for testing. The authors introduce SDNshield, a solution that employs Network Function Virtualization (NFV) and has three stages for overload control:

1. It utilizes statistical methods to differentiate between legitimate and malicious flows using a Bayesian metric obtained from the packet-in messages.
2. The system performs TCP verification to ensure that legitimate flows acquire the necessary resources, mitigating the false positives and negatives from the previous stage.
3. Given the fact that legitimate flows typically consist of numerous continuous packets that persist for a considerable duration, the solution introduces parameters to identify the flows and restore the connection to the controller.

Table 2 summarizes the attacks reviewed in this section. The table reveals a variety of controller platforms, with Ryu and Floodlight receiving the most attention. Additionally, while several detection parameters are used, attack identification predominantly relies on OpenFlow traffic analysis.

The primary detection parameters are statistics related to the SDN obtained through OpenFlow and the quantities or probability distribution of the Packet-in messages. The disruption of these elements usually indicates a modification in the network due to some of the most common DDoS attacks.

Other mechanisms, such as using a database or parameter map of network elements, are also instrumental in the detection process. This allows the controller to compare the feasibility of an event before accepting it.

Authors usually consider the CPU and memory use as key indicators for performance metrics. However, they rely on their methods to specify the performance of their proposals, such as the algorithms used.

Studies using the Ryu, Floodlight, and POX controllers present various metrics. However, ONOS presents only one metric, raising questions about its ease of use and the simplicity of obtaining statistics with this controller.

**Table 2** Controllers used in DDoS attacks and detection

| Controller | Study | Detection parameter | Performance measure of the proposal |
|---|---|---|---|
| Ryu | [17] | Deep learning and network parameters counters | CPU and memory use, throughput |
| | [18] | Table-miss count | Continuity of network operation |
| | [19] | Performance index in the controller | – |
| | [20] | Packet-in messages counted by external server INSPECTOR | TCAM metrics, CPU and memory use and openflow channel load |
| | [21] | | CPU use, response time, flow request and control channel bandwidth |
| | [22] | Flow installation | Time attack pattern |
| | [23] | Packet-in count to calculate Jain's index | Fairness values with different attack counters |
| | [15] | NA | Response time and CPU use |
| Floodlight | [24] | entropy variations of source IP addresses and IP packet-sizes | Amount of packets to locate the bot |
| | [25] | Types of Packet-In messages for classification | CPU and memory use |
| | [26] | Packet-in, Packet-out, Flow-mod and TCP-ACK messages | Performance of ML method in dataset |
| | [27] | Traffic distribution for imbalances between high and low traffic flows | Performance of ML method in dataset |
| | [28] | Map of switching, hosts and identification of injection behaviours | CPU use and performance of ML method in dataset |
| | [29] | Probability density from Packet-in sampling | Quantity of Packet-in messages |
| POX | [21] | Attack patterns in a database to match with current flow | Response time, CPU use, Control channel bandwidth |
| | [30] | Flow and packet parameters to a fuzzy c-means algorithm | CPU use and performance of algorithm |
| | [31] | Rate of Packet-in messages arriving to the controller | Traffic overhead |
| ONOS | [32] | Flow statistics from SDN | Performance of ML method |

The following subsections present the studies specifying the controller utilized in their proposed methods and experiments.

### 3.1.1 Ryu Controller

Among the most recent investigations is the study by Cherian et al. [17], which explores a Distributed Denial-of-Service (DDoS) attack in IoT networks. In this scenario, the attacker attempts to induce numerous "table-miss entries" within the switch, thereby generating a significant flow of "packet-in" messages. The countermeasure proposed by the controller in this study involves utilizing a counter values for different network parameters to detect the attack as well as deep-learning techniques.

The authors in [18] examine an Intrusion Detection System (IDS) using SNORT. They propose an algorithm that detects Distributed Denial-of-Service (DDoS) attacks using a "table miss-entry" approach, which applies a preliminary filter to packet-in messages.

Moreover, authors in [19] experiment with SYN flood attack that targets a server in the network but affects the controller directly with a packet-in flooding. The detection mechanism relays on the performance index of the controller, which is reviewed constantly.

In [20], the authors propose a defensive strategy executed by a server dubbed INSPECTOR within a Fat-tree topology. This server examines all packet-in messages, maintaining a database of authenticated hosts (including switch ports, IP addresses, MAC addresses, etc.). The system either allows or drops flows based on consistency checks. The authors report better performance than previous studies. However, while the authors claim to use a proprietary dataset, they do not disclose how it was assembled.

The study by [21] puts forward a parallel flow installation model, whereby rules are installed on all switches included in a specific flow. This reduces the volume of packet-in messages. Nonetheless, it does not propose a specific method to detect or mitigate DDoS attacks.

The authors in [22] introduce a Tree network topology for experimentation. They employ the Support Vector Machine (SVM) method to detect abnormal traffic, using a constructed dataset that includes information extracted from packet-in messages, such as source and destination IP addresses and ports.

The authors of [23] propose a DoS attack via ICMP flooding, which produces high packet-rate flows. Nevertheless, they suggest that their proposed detection method, which is statistical, could be utilized for low-rate flow attacks as well.

Finally, the authors in [15] suggest modifying the Black Nurse attack, where a firewall becomes a DDoS victim through receipt of excessive Internet Control Message Protocol (ICMP) error messages. Here, the attack is directed at the controller, as there is a botnet consisting of ICMP message senders and receivers, with the latter responding with errors. Consequently, the controller must search for alternative routes to the destination, even when the current ones are still valid. The attack is designed to transmit a low-rate flow, obstructing detection in the network, but capable of disrupting the controller in a short time.

### 3.1.2 Floodlight Controller

As the first example, the study in [24] present an implementation of a cloud-based environment in Amazon Web Services (AWS) using ODL, Flowvisor and mininet to detect and mitigate low rate DDoS that attack the network web servers. The Flowvisor is used to slice

the cloud part of the experiment, to make it similar to cloud environments. Authors utilize Low Orbit Ion Canon (LOIC) tool for the attack,generating repetitive packets from each bot.

In [25], the authors conducted a study on packet-in flooding to the controller, focusing on network discovery rather than a deliberate attack. The proposed approach for protection involves implementing a Packet-in filtering mechanism that classifies Packet-in messages and selectively forwards only the essential ones to other core controller modules. This paper introduces a new module within the architecture of the Floodlight controller to mitigate Packet-in flooding by filtering packets based on three categories: flow setup, state change, and forward (of lesser importance). In their work [26], the authors make a significant contribution by evaluating various time windows for detecting attacks using machine learning techniques. The importance of this evaluation lies in finding the optimal window size that balances the risk of overlooking saturation attacks if the window is too long and the risk of insufficient information to identify an attack if the window is too short. Furthermore, the paper explores the impact of saturation attacks on the rate of *packet_in* and *packet_out* OpenFlow messages, as well as the relationship between attack types and their corresponding behaviors. The datasets employed in this study incorporate features such as the number of *OFPT_PACKET_IN*, number of *OFPT_PACKET_OUT*, number of *OFPT_PACKET_MOD*, and number of *TCP_ACK*. While the results include tests conducted with KNN, NB, and SVM, the paper lacks clarity on the dataset size and the ratio between training and testing samples. On the other hand, Cui et al. [27] proposes the use of the k-means method as an unsupervised algorithm to detect unbalanced traffic. This approach aims to reduce the load on the controller during packet-in flooding. The proposed architecture consists of a detection module (traffic dictionary) and a filtering module. The detection method is based on analyzing the traffic distribution for imbalances between high and low traffic flows. Another compelling study is showcased in [28], where the authors introduce the Packet Injection Exploiting Attack. This attack consists of a malicious host transmitting forged packets within the data plane, complete with spoofed addresses, to mislead the controller into believing new hosts are in the network. This type of attack will be called “Host Injection” throughout this survey. Finally, the research conducted in [29] investigates the packet-in flooding attack utilizing IP and MAC spoofing and presents a detection method that employs a statistical threshold for the number of packet-in messages. The proposed method considers the Poisson distribution of packet-in arrivals. The proposed system sends flow_mod messages to the switches as a defensive measure to halt the attack.

### 3.1.3 POX Controller

Though the authors in [21] utilize the Ryu controller in their study, they also present their solution in conjunction with POX. Moreover, the paper [30] proposes a method to detect and mitigate DDoS attacks directed at the controller via packet-in message flooding. The system comprises four elements: a switch classifier (assigns a probability to switches, identifying them as either malicious or normal), a feature extractor (calculates six parameters of the flows to input into the anomaly detector), an anomaly detector (employs a fuzzy c-means algorithm for classification into normal or attack flow), and an attack mitigator (a module designed to set up rules for the switches). The reported experimental results demonstrate low CPU overhead coupled with high detection accuracy. Authors in [31] present a statistical anomaly detection method detailed in 4.1 but with attacks related to DoS of three classes: TCP-SYN, UDP, and ICMP.

#### 3.1.4 ONOS Controller

Perez-Diaz et al. [32] was the only proposal that considered DDoS attacks that used an ONOS controller. In that study, authors present a flexible security architecture in SDN that detects low rate DDoS attacks using machine learning.

### 3.2 Topology Poisoning

Contrary to Denial-of-Service (DoS) attacks, topology poisoning attacks typically generate low-rate flows within the network, necessitating different detection mechanisms. Apart from the study in [33], which focuses on ARP table poisoning in switches, the attacks outlined in this section exploit vulnerabilities inherent in the OpenFlow discovery protocol, which is based on the Link Layer Discovery Protocol (LLDP).

Table 3 presents an overview of research efforts addressing topology poisoning attacks across various SDN controller platforms. Like DDoS attacks, Floodlight is the predominant controller used for testing, while LLDP packet probing and verification are the most common methods for attack detection. Due to the specificity of an attack in which the attacker aims to falsify links, the detection mechanisms rely heavily on validation or statistics of LLDP packets. This reliance on LLDP packets, a proven and effective method, provides confidence about the robustness of the detection mechanisms. Similarly to the studies of DDoS attacks, CPU and memory use are standard performance metrics in topology poisoning attacks. Researchers typically use Floodlight and Ryu controllers, with others having only one study. The following sub-sections present the different studies included in the review.

#### 3.2.1 Floodlight Controller

The authors in [34] propose the MLLG system, which employs a machine learning classification model to validate Link Layer Discovery Protocol (LLDP) packets before a topology update. They utilized six classifiers to train and test with the dataset collected during their experiments, with the best results achieved using the random forest classifier.

On the other hand, Gao and Xu [35] explores the host-hijacking attack, in which a malicious host masquerades as a legitimate host within the network. As a defensive measure, the authors propose a process for verifying the legitimacy of host migration. This involves two steps: confirming the receipt of a "port down" message and verifying that the previous location is unreachable. For link fabrication attacks, the paper introduces the concept of information entropy to assess the distribution of destination IP addresses in LLDP frames within the SDN.

Furthermore, Shrivastava and Kataoka [36] underscores the advantages of hybrid SDN, such as cost-effectiveness compared to an abrupt transition from traditional networking to SDN. The authors uncover a new attack dubbed multi-hop link fabrication, which disrupts network operations and impairs applications such as Quality of Service (QoS) and load balancing. Such attacks are categorized as Topology Poisoning attacks and can potentially trigger more severe threats, such as Denial-of-Service (DoS) at the controller. As a mitigation strategy, the authors propose a method known as hybrid-shield, capable of identifying

**Table 3** Controllers used in topology poisoning attacks and detection

| Controller | Study | Detection parameter | Performance measure of the mitigation technique |
|---|---|---|---|
| Floodlight | [34] | LLDP packets validation before topology updates | Performance of ML method |
| | [35] | Delay threshold on the LLDP relay and information entropy of the destination IP | False positive rate and detection time |
| | [36] | ARP responses to validations from controller to identify attacker | Attack detection time and accuracy |
| | [37] | N/A | N/A |
| | [38] | Port id already registered in the controller databased compared against the received in LLDP packets | Packet loss percentage |
| Ryu | [39] | Statistics in each switch that allow identification of abnormal processes | CPU usage, storage overhead, time overhead |
| | [40] | Prevention using moving target defense | N/A |
| ONOS | [41] | Variance-based anomaly with a comparison of the LLDP probing packets | CPU use, Memory use, Control channel overhead |
| Opendaylight | [42] | Arrival of LLDP probing packets | CPU use, Memory use, packet loss, througthput, relay time |
| POX | [33] | Source address of ARP packets that require a new flow-rule | CPU use, Packet delivery rate, Network througtput |

N/A indicates no detection or mitigation techniques presented in the proposal

whether an attack has been launched and preventing its consequences. This method includes behavior monitoring and validation of the MAC learning process.

The paper [37] highlights a significant vulnerability—a persistent compromise of the switch can be achieved simply by adding a second, malicious controller (a feature available in OpenFlow 1.2+). To accomplish this, the authors utilized a secondary malicious controller employing POX software. The identified attacks include: (1) Basic: False link between two compromised switches. (2) Neighbor: False link between neighbors of two compromised switches. (3) Merging: Two or more compromised switches appearing as a single entity. (4) Invisible Switch: A single switch can use the neighbor attack to "disappear" from the controller's view. The authors tested "fully deterministic" and "load balancing" routing across eight types of topologies, such as binary tree, fat tree, hypercube, and others. Proposed defensive strategies include architectural and routing methods, static detection, dynamic detection, and hardware-based solutions. The paper does not propose a detection strategy, but try to avoid the attack with the topology of the network.

In [38], the authors examine SDN from the perspective of network management, identifying the main vulnerability as the semantic gap, where there is no specification for information integrity or ownership tracking mechanism. This viewpoint is significant given the extensive implementation of SDN controllers for network management. The paper presents attack mechanisms affecting availability, integrity, and confidentiality, demonstrating the vulnerabilities of the architecture.

### 3.2.2 Ryu Controller

In [39], the authors present the Invisible Assailant Attack (IAA), which consists of 14 attack phases and can mantain fake links even with multiple defense strategies implemented simultaneously. The authors propose a Route Path Verification (RPV) mechanism to circumvent IAA. This strategy employs an innovative method to disguise the 1125 attack packets as normal network packets already present in the network, significantly increasing the difficulty of detection. The IAA considers the injection and maintenance of a false link for a link fabrication attack. The RPV method allows to track the packet's source back to its origin, following the statistics in each switch. This process enables the identification of abnormal processes if the RPV cannot identify the in-port of a given packet that generates a topology modification.

Moreover, the authors in [40] do not specify the mechanism that poisons the controller's view of the network. Instead, they contribute by introducing Moving Target Defense (MTD) techniques, positing that a static network (consistent IPs and MACs) is more vulnerable to poisoning attacks. The MTD complicates leaking real IPs that could be used in an attack.

### 3.2.3 Other Controllers

The authors in [41] propose a defense mechanism leveraging the P4 programmability for switches. Topology poisoning is a particularly insidious attack for Software-Defined Networks (SDN), owing to its stealthy nature and potential to disrupt operations. These attacks are typically executed by manipulating the Link Layer Discovery Protocol (LLDP). The authors propose a two-fold solution involving source address verification and in-network anomaly detection. The detection methodology is based on the variance in LLDP intervals to identify anomalies. Researchers used ONOS as controller. The study presented in [42]

proposes a probing mechanism to detect fake links created by fabricated Link Layer Discovery Protocol (LLDP) packets using three methods:

1. Relaying LLDP packets using hosts,
2. Injecting LLDP packets,
3. Relaying LLDP packets using switches.

The proposed methodology involves the following steps:

1. Monitor updates to the network view,
2. Generate a probing packet in the switches with changes,
3. Install a flow in the switches.

The proposed detection mechanism is a probing system that generates and sends packets to verify legitimate links and detect fake links, as well as the malicious switches responsible for them. This method has proven to be sufficiently fast for implementation in an operational network. The authors employed the OpenDaylight controller for this study. Finally, the authors in [33] propose a scenario involving IoT with SDN, where attackers poison the ARP tables and the SDN controller, potentially leading to unauthorized access to network resources. The authors suggest a solution in which a server acts as a mediator for the SDN controller, handling address translation. This mediator employs a historical table with corresponding IP to MAC lists and discards any packet that does not match. If multiple non-matching packets are sent, the port of origin is blocked.

### 3.3 Bandwidth

The bandwidth attack aims to saturate a network link, often targeting the southbound interface (data to control plane) of the SDN architecture.

Table 4 presents bandwidth and saturation attacks. The limited number of studies suggests this is an under-explored area in SDN security research, highlighting opportunities for future investigations across different controller platforms.

Only four studies were included in the survey that considered bandwidth attacks only, not as part of a DDoS, and once again, Ryu and Floodlight are the preferred controllers. The metrics are dissimilar since the attacks have different purposes, such as saturating shared control channels or the one connecting to a network resource. However, performance measurements are, once again, related to resource use.

The first study [43] underscores the need for a mitigation strategy against data-to-control plane saturation attacks, in which the response time decreases without significantly impacting the remaining network flows. As these types of attacks often generate a high volume of invalid table entries in the switches, the challenge lies in the lack of effective methods for cleaning up bad attack flows post-detection. The paper proposes a linear discriminant analysis method to calculate a feature called control channel occupation rate, which allows the detection of the occurrence of an attack.

Additionally, in [44], the authors employ the Floodlight controller and analyze the self-similarity of OpenFlow traffic (including OpenFlow traffic flows and the ratio of the number of OFPT_PACKET_IN packets to OFPT_PACKET_OUT packets) to detect anomalous flows. They employ the Hurst exponent to distinguish between normal and malicious traffic. The experiments were conducted in physical and simulated environments, with datasets

**Table 4** Controllers used in bandwidth attacks and detection

| Controller | Study | Detection parameter | Performance measure of the mitigation technique |
|---|---|---|---|
| Ryu | [43] | Control channel occupation rate from linear discriminant analysis | Buffered packets, network recovery delay |
| Floodlight | [44] | Self-similarity of OpenFlow traffic measured by Hurst exponents | CPU use and performance of algorithm |
| | [45] | Delay presented by control messages in shared channels | Link use, port use |
| | [46] | Statistics of network flows | Available bandwidth, performance of ML method |

created by collecting OpenFlow traffic with varying traffic parameters such as packet sizes and protocols. The authors used the Distributed Internet Traffic Generator (DITG) for normal traffic. Various techniques such as spoofing, port scanning, and flooding were used to generate an environment saturated with anomalous traffic, constituting 63 different attacks. The tools used for this were HPING3 and LOIC. Using both Hurst coefficients and a threshold determined through various experiments, the authors could detect anomalous traffic with up to 97% accuracy.

Authors in [45] elucidate a vulnerability inherent in SDN controllers situated in single locations that rely on data channels to connect disparate switches. This shared link scenario opens up a potential attack surface. The vulnerability is exposed through reconnaissance, which checks for channel delays. When a link is shared, the delay is demonstrably longer.

Lastly, Rasool et al. [46] delineates an attack strategy whereby an intruder detects the path to a network resource and introduces fraudulent servers into the path. A botnet then leverages these false servers to generate low-rate TCP traffic. Despite the seemingly innocuous traffic rate, the number of bots involved results in path saturation.

### 3.4 Other Attacks

The studies [13, 14, 47] present attacks that do not fit precisely in the defined categories. The study [13] explores the possibility of launching a Man in the Middle MITM attack directed to an SDN controller and the impact on the network, explicitly considering a remote controller connected through a WAN. The attack is carried out through ARP poisoning using Kali Linux in the controller's same IP segment. On the other hand, the authors in [14] introduce an Advanced Persistent Threat (APT) model against the SDN controller, encompassing six distinct attacks executed methodically and organized. Nevertheless, the paper does not implement a series of steps that align with the established definition of an APT, such as the one presented in [48], where it is defined as a stealthy group of attackers aiming to steal a target's data without being detected.

Table 5 shows the two remaining attacks considered in this review. Finally, Dixit et al. [47] offers an analysis, instead of proposing an attack or defense mechanism, highlighting the vulnerabilities of SDN about the semantic gap between the control and data planes.

### 3.5 General Considerations

The findings of this survey focus on three major elements regarding attacks against the SDN controller: the network parameters used to monitor the traffic, the performance metrics of the mitigation proposal, and the controller preference and usage. For DDoS attacks, the robustness of the primary detection parameters, including OpenFlow statistics and

**Table 5** Controllers used in other attacks and detection

| Attack | Controller | Study | Detection parameter | Performance measure of the mitigation technique |
|---|---|---|---|---|
| MITM | Opendayligth | [13] | N/A | N/A |
| APT | Floodlight | [14] | Correlation of the attack behaviors to identify the attack tree | Performance of algorithm |

N/A indicates no detection or mitigation techniques presented in the proposal

Packet-in message distributions, is a reassuring sign. Disruptions in these parameters serve as strong indicators of potential attacks. Additionally, the use of databases or parameter maps of network elements adds another layer of validation before acceptance. In topology poisoning attacks, the importance of LLDP packet statistics and validation for detecting falsified links further demonstrates the robustness of these methods. Moreover, CPU and memory usage are standard performance metrics across different attack types, indicating a controller's ability to handle network stress. For DDoS attacks, Ryu, Floodlight, and POX controllers present various metrics, while ONOS presents only one, raising questions about its ease of use and statistic accessibility. Floodlight and Ryu are predominantly used in topology poisoning attacks, maintaining consistent performance metrics. Bandwidth attack studies also focus on resource usage, despite varied metrics for different attack purposes. As per the controller platform, Ryu and Floodlight are preferred across studies, probable for their comprehensive metrics and usability. ONOS and Opendaylight are the least favored.

## 4 Detection Methods

The second research question concerns effective strategies for detecting the aforementioned attacks. The reviewed studies identified two main groups of detection methods: Statistical Anomaly Detection and Machine Learning Algorithms. The distribution across both categories is balanced, though not all papers specify or propose a detection method. Certain studies, such as [15], focus solely on proposing an attack, while others, like [47], present an analytical perspective. Figure 4 provides a classification of the detection methods identified.

The following sections delve into the principal aspects of the proposals for both methods and include an additional section that considers a third category of "other" proposals for detection.

### 4.1 Statistical Anomaly Detection

This type of detection uses different statistical parameters such as randomness, similarity, or thresholds. Table 6 presents the metrics and performance used in these studies, along with those of the machine learning approach. The detection methods are based on statistical tools, which establish their value based on diverse metrics, mainly accuracy. However, true positive rate, precision, and false negative rate are part of the classification metrics. Also, studies consider time for different activities, such as setting up a connection, a flow, or detecting an incident. The reported performances in this scenario are all over 90%, confirming the efficiency of the presented methods. Additionally, the detection times range from less than 1 s to a maximum of 10 s.

In [24], authors propose to use an entropy measurement of the traffic that has been captured using a sniffer. The algorithm detects the randomness associated with the IP address and packet size. The authors assume Gaussian and Poisson distributions for the analysis in benign traffic, which they will compare against the capture. After the detection, the controller applies a DENY rule to the source IP in the switch. The reported time for detection in the paper is from 2 to 6 s.

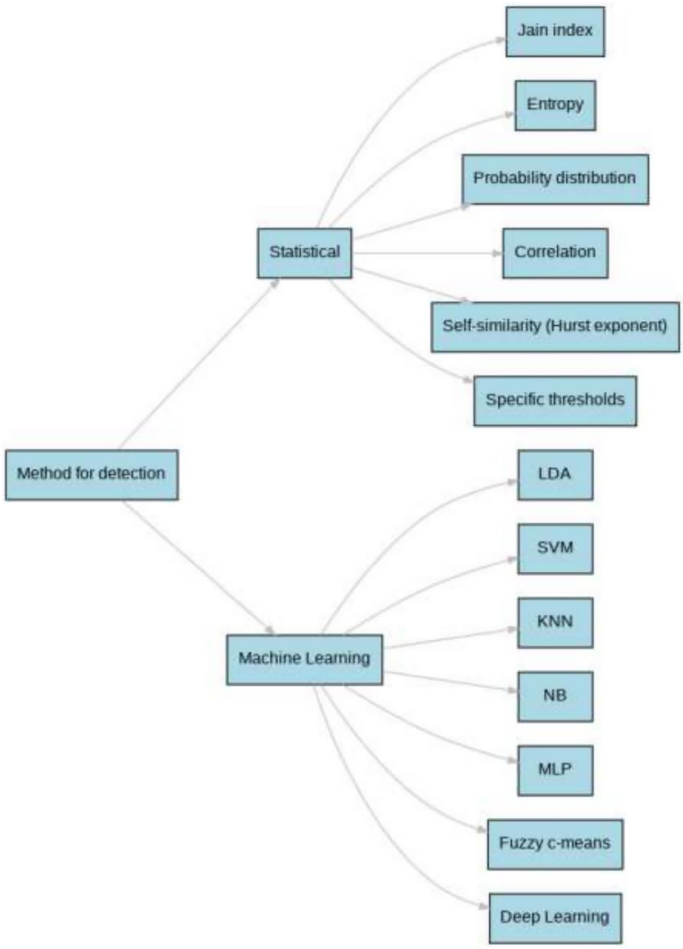



**Fig. 4** Detection methods found in the survey for attacks directed to the controller

A similar approach is used in [29], which proposes to establish a threshold for the probability density of the number of packet-in messages, considering the Poisson distribution of the packet-in arrival.

The study in [31] presents a method to detect the attack with a statistical profile of the traffic for the packet-in at the controller. The parameter PacketInRate (PIR) is selected dynamically to define a threshold to detect the attack. The method is protocol-independent.

Two steps for detection are used in [19]. First, a module checks the controller's performance with eight different metrics to assess if there is an ongoing SYN attack. The second stage uses historical values of those metrics to identify the type of attack (IP spoofing, MAC-IP spoofing, or flash crowd). The AEGIS proposal has additional statistical indexes to confirm if the network is under a SYN flood attack.

The type of attack presented in [41] is topology, and it is based on LLDP vulnerabilities. For the detection of fake news links, authors detect the interval variance of the LLDP packets. If the variance is higher than a predefined tolerance, the link is not accepted as valid.

**Table 6** Detection methods—standardized metrics

| Study | Attack | ML technique | Accur. (%) | FPR/FNR (%) | Detection time | Other metrics |
|---|---|---|---|---|---|---|
| [24] | Low-Rate DDoS | Entropy measurement | 97.6 | FPR: 3.18% FNR: 3.29% | 14.45 ms–10.02 s | N/A |
| [29] | Packet-In Injection | Probability density of the number of packet-in messages | N/A | TPR: 92–98.2% | N/A | N/A |
| [31] | Saturation | PacketInRate (PIR) | N/A | N/A | 0.1 s | 0% overhead |
| [19] | TCP SYN Flood | 8 different metrics | 97.78 | N/A | N/A | TCP setup time: 63.7 ms |
| [41] | Topology Poisoning | interval variance of the LLDP packets | N/A | FPR: 5% FNR: 5% for 500 smp | N/A | 500 samples req |
| [16] | DDoS | Statistical Differentiation and TCP conn. verification | FNR: 0 | 10ms | Flow setup delays | |
| [45] | Link Disruption | T-test | 98% | FPR: 4% | 3 s for 12 rules | N/A |
| [14] | APT | correlation | 90% | N/A | less 0.2 s | N/A |
| [23] | Flooding | Jain's index and entropy | N/A | N/A | N/A | Eff: less than 10% attackers |
| [43] | Saturation | Linear Discriminant Analysis | 84% | N/A | N/A | Recovery delay |
| [22] | Packet-in flood | SVM | N/A | N/A | N/A | Reduced effects of 36% |
| [26] | Saturation | SVM, NB, K-NN | N/A | N/A | N/A | Up to Precision:97% Recall:99% F-1:98% Time-window of traffic analysis: 3 min |
| [46] | Link Flooding | ANN | 85% | N/A | N/A | Recall:95% F1:76% |
| [27] | TCP/ICMP flood | K-means | N/A | 4.5 to 6.5 s | 6 s | Recall:90% |
| [32] | Low-Rate DDoS | J48, Random Tree, REP Tree, RF, MLP, and SVM | 95.01% | FPR: 0.52% | N/A | N/A |
| [34] | Link Latency | LR, SVM, K-NN, NB, RF, MLP | 98.22% | TPR:0.97% | N/A | Precision:93.24% F1:92.99% |
| [30] | Packet-In flood | Bayesian Network and fuzzy c-means | N/A | TPR: 95.68% | N/A | |

N/A indicates metric not reported in original study. Studies often prioritize different metrics based on their attack scenarios and detection goals

On the other hand, Chen et al. [16] presents three stages to take countermeasures against traffic; the first step is a statistical method, while the others are for verification of the classification of the previous one.

The study [45] proposes a detection method called adversarial path reconnaissance with a probing part to estimate the delays for control messages for shared links (both data and control). The detection part of the method uses a statistical metric called the T-test that establishes the possibility of two groups of samples being in the same group. The difference allows the method to identify anomalies.

In [44], authors propose to use a hurst exponent to review the self-similarity of the traffic since saturation attacks usually have higher values. On the contrary, Shan-Shan and Ya-Bin [14] proposes to collect data from different points and calculate a correlation degree between normal and attack scenarios. Finally [23] uses the Jain's index and entropy for the detection of DoS attacks

### 4.2 Machine Learning Algorithms

The advancement in machine learning techniques, coupled with increasing computational power, has facilitated the proliferation of studies utilizing machine learning for anomaly detection, as long as the datasets for model training are available. Table 6 shows that traditional ML methods (SVM, NB, K-NN, RF) are used more frequently than neural networks (ANN, MLP), although both approaches show high performance, achieving over 90% accuracy. Accuracy is the most commonly reported metric, often exceeding 90%. However, some studies use more comprehensive metrics like F1-score, which balances precision and recall. Regarding Detection Speed, only [27] explicitly reports on detection time (6 s for attacks over 200 pps). This suggests that while accuracy is well-addressed, real-time detection capabilities may need more focus.

Within the spectrum of machine learning methods for anomaly detection, there have been proposals to utilize various techniques, such as Linear Discriminant Analysis (LDA), as described in [43]. This method detected a bandwidth attack in the control channel using a small dataset of 192 attacks. The detection scheme monitors not only packet-in but also the occupation of the control channel. Upon detection, the system identifies the bots and activates a mechanism to preserve benign flows and remove bad traffic.

Another example is [22], which uses a Support Vector Machine (SVM) from a dataset created from packet-in messages, such as source and destination IP address and source and port.

The work presented in [26] is notable due to the various time windows evaluated by the authors to identify the optimal one. In this study, detection is achieved by classifiers such as SVM, K-NN, and NB, trained with datasets featuring metrics related to OpenFlow messages, such as OFPT-PACKET-IN, OFPT-PACKET-OUT, OFPT-PACKET-MOD, and TCP-ACK.

On the other hand, Rasool et al. [46] suggests detecting anomalies by monitoring the control channel and presenting traffic flow statistics to a processing module handling flood detection. The module then implements a Multilayer Perceptron Algorithm to identify anomalies.

Authors in [27] propose the use of the k-means algorithm in an unsupervised manner to detect unbalanced traffic. In contrast, Perez-Diaz et al. [32] presents six different machine learning methods, namely J48, Random Tree, REP Tree, Random Forest, MLP, and SVM, to identify DDoS attacks. Also, soltani2021link et al. [34] proposes different methods for

the detection of anomalies: Logistic Regression (LR), Support Vector Machine (SVM), K-Nearest Neighbors (K-NN), Naive Bayes (NB), Random Forest (RF), and Multi-Layer Perceptron (MLP).

Finally, Gao et al. [30] presents a comprehensive approach consisting of four modules for detection. One module is the switch classifier, which assigns a probability to switches to identify them as either malicious or normal. Another module is the feature extractor, which calculates six parameters of the flows to be used in the anomaly detector. The anomaly detector utilizes the fuzzy c-means algorithm to classify flows as normal or attack. Lastly, an attack mitigator module is responsible for setting up switch rules.

### 4.3 Others

The proposals falling outside the two main categories presented are related to topology poisoning attacks involving fake links and host injection. These attacks exploit the vulnerabilities of the OpenFlow Discovery Protocol (OFDP), which is based on the Link Layer Discovery Protocol (LLDP).

One such study, Alimohammadifar et al. [42], proposes a detection method involving probing legitimate links by generating and sending packets to assess behavior. However, this method may disrupt the normal network operation and might be impractical during periods other than topology modification, as noted in [37].

A second notable method, introduced in [38], suggests collecting details from LLDP packets and comparing them to an existing database of port and switch IDs. If any inconsistency is detected, the controller can flag the packet as malicious and exclude it from topology updates.

Similarly, Li et al. [28] suggests creating and maintaining a mapping table between switches and hosts. This table is used to verify the legitimacy of a new host in the event of a host injection attack.

### 4.4 General Considerations

The literature review provided two broad categories for detecting attacks in SDN networks: statistical-based and machine learning-based techniques. Statistical-based detection methods rely on parameters such as randomness, similarity, or thresholds. These methods establish their effectiveness based on diverse metrics, primarily accuracy. However, the literature also presents other classification metrics like true positive rate, precision, and false negative rate. Moreover, time-based metrics for various activities, such as connection setup, flow establishment, or incident detection, also appear for evaluating these methods. On the other hand, machine learning-based detection methods employ a range of techniques, from traditional algorithms like Support Vector Machines (SVM), Naive Bayes (NB), K-Nearest Neighbors (K-NN), and Random Forests (RF) to more advanced approaches such as Artificial Neural Networks (ANN) and Multilayer Perceptrons (MLP). These methods are evaluated using metrics similar to those of statistical approaches, with accuracy being the most commonly reported.

The metrics presented by each author varied, which makes the direct comparison challenging. However, the comparative analysis of statistical and machine learning-based detection methods, shown in Table 6, reveals key factors contributing to their relative effectiveness. Statistical methods generally achieve faster detection times, often within 1–10 s, with high accuracy levels exceeding 97%, except for [14] which reports 90%. These approaches

**Table 7** Comprehensive classification of SDN defense actions

| Defense type | Count | References |
|---|---|---|
| Flow/Packet discard | 14 | [16–18, 24, 29, 31, 34, 35, 41, 46, 49–52] |
| Rule-based filtering | 8 | [20, 25, 27, 38–40, 42, 43] |
| Traffic management | 7 | [21, 28, 30, 32, 33, 45, 53] |

Count represents the number of reviewed papers implementing each defense type

are well-suited for real-time detection scenarios where speed is critical. On the other hand, machine learning techniques demonstrate superior performance, reaching 95–98% accuracy, when dealing with more complex attack patterns such as sophisticated DDoS. However, the computational overhead of ML models can result in longer detection times. For example, in cui2020ddos, authors report needing 6 s to detect attacks over 200 packets per second, and in [26], they declare a proper time-window of 3 min for OpenFlow traffic analysis for detection. The analysis also suggests a trade-off between detection speed and comprehensive performance metrics, as statistical methods tend to prioritize raw accuracy while ML-based approaches provide deeper insights through metrics like precision, recall, and F1-score. It is noteworthy the lack of combined detection methods. Out of the 18 studies, only two of them considered more than one stage for detection, leaving a high risk of getting false positive or false negative events. Overall, the choice of detection mechanism depends on the specific requirements of the network environment, the type of attacks being targeted, and the desired balance between real-time responsiveness and advanced analytical capabilities.

## 5 Defense Actions

The literature review identified three main defense action categories for the attacks directed at the controller, as presented in Fig. 5. Each category is described below:

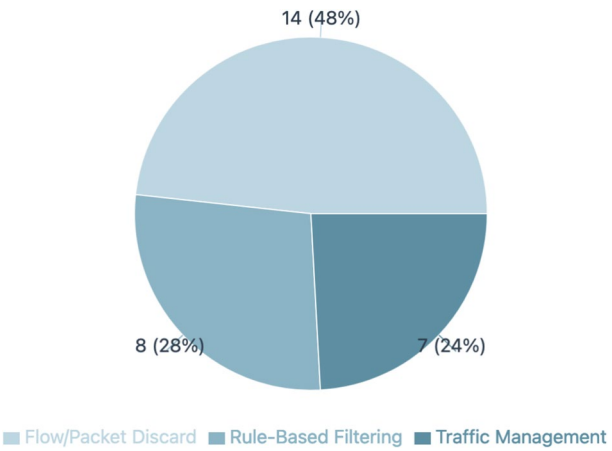



**Fig. 5** Distribution of SDN defense actions

1. Flow Discard: Involves the controller or an auxiliary server, discarding specific flows or packets that are marked as malicious or suspicious by a previous detection step.
2. Rule-Based Filtering: Proposals that implement rule-based filters or policies to filter and control network traffic based on predefined criteria. These rules are applied by the controller to switches to allow legitimate traffic while blocking or dropping packets associated with attacks.
3. Traffic control: Encompasses techniques aimed at managing and controlling the flow of network traffic during security incidents or attacks. It involves a combination of traffic blocking and diversion strategies to ensure effective defense and protection.

Table 7 presents a classification of the different studies by defense action.

The research conducted by Ravi et al. [19] transcends a singular classification, as it outlines two mitigation strategies, with each designed for a specific type of attack. The study proposes the following methods: (1) implementing drop rules for MAC addresses when the IP count surpasses half the defined period and (2) initiating random timeouts for drops in conjunction with a gradual elongation of the standard processing period, thereby assuring legitimate users are capable of retrieving the necessary information.

The analysis of defense actions against SDN controller attacks reveals three primary categories implemented across the surveyed literature. Flow/Packet Discard emerged as the most commonly implemented defense mechanism, with 14 studies (48%) utilizing this approach. This preference might be attributed to the straightforward method of removing flows from the flow-table. However, it is important to note that these categories are not mutually exclusive. Additionally, this categorization simplifies complex defense mechanisms that might incorporate elements from multiple categories, potentially underrepresenting the sophistication of some solutions.

## 6 Controllers

About the third research question, which addresses the controllers employed in the studies, our findings reveal that the most commonly utilized controller in the reviewed studies is Floodlight, followed by Ryu, Pox, ONOS, and ODL, as depicted in Fig. 6.

It's important to note that the total number of studies included does not correspond to the sum of the controllers depicted in the figure. This discrepancy is due to [47] experimenting with both ODL and ONOS, while [21] involves both POX and Ryu. Also, Agrawal and Tapaswi [24] uses both ODL and Flowvisor. Conversely, Chen et al. [16] does not specify which controller was utilized. A brief definition of each controller is presented in Table 8.

As detailed in [54], while the literature mentions over 30 controllers, only a handful offer the necessary implementation tools, such as code and documentation. Of the eight controllers (including NOX, Beacon, OpenMul, and Maestro) discussed in the study, only six are in active use, as identified in this survey.

When writing this survey, only ONOS and ODL have updated websites and appear to be active products. Additionally, as sources such as [4] suggested, ODL and ONOS could significantly streamline the transition to 5G. Notably, the controllers that hold the potential to facilitate the adoption of emerging technologies are, somewhat paradoxically, the least employed in the studies reviewed. Both of these controllers are based on the OSGI

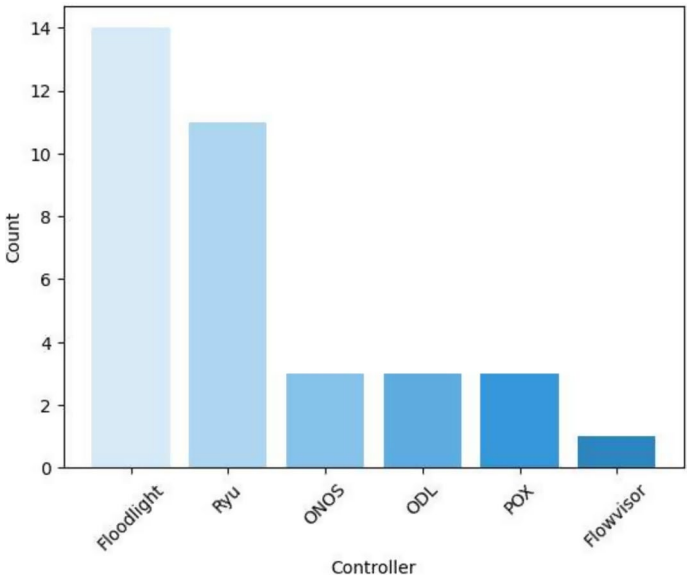



**Fig. 6** Controller usage in the reviewed studies

/ Apache Karaf software architecture, which renders them highly modular and flexible, albeit at the expense of a potentially steep learning curve.

Table 8 provides insights into the types of attacks studied across various SDN controllers. DDoS and topology poisoning attacks emerge as the most commonly investigated threats, appearing in studies across multiple controller platforms. Bandwidth overload is being examined in Floodlight and Ryu environments. The table reveals some specialized focus areas: Advanced Persistent Threats (APT) are uniquely studied in Floodlight, while Man-in-the-Middle (MITM) attacks are exclusively explored in ODL (OpenDaylight) studies.

## 7 Datasets

The pool of datasets used for anomaly detection in network traffic is relatively limited. The study presented in [55] highlights the primary datasets employed in SDN proposals, including DARPA 99, KDD-Cup 1999, LongTail, NSFNET topology, NSL-KDD, and UNB ISCX. In addition, this survey includes the CIC-IDS-2018 dataset in the review.

The CIC-IDS-2018 dataset, the most recent among these, is a significant addition to the field. However, it neither considers the most aggressive attacks nor is it specifically designed for SDN scenarios.

Reviewing the literature, it is evident that traditional datasets, which cannot replicate the plane separation operation of the SDN architecture, have been the standard. Table 9 details these outdated datasets, highlighting the critical need for new, more effective datasets in SDN research.

Nevertheless, some researches have created datasets in SDN environments using different approaches. Section 7.1 presents a review of SDN datasets

**Table 8** Use of controllers for studies on attack to the controller

| Controller | Description | Studies | Type of attacks |
|---|---|---|---|
| Floodlight | Open-source SDN controller developed by a community. It is primarily written in Java | 14 | DDoS, Topology, Bandwidth, APT |
| Ryu | Modular framework for SDN, developed in Python | 11 | DDoS, Topology, Bandwidth |
| POX | Python OpenFlow controller that evolved to support related functions | 3 | DDoS, Topology |
| ODL | Developed by The Linux Foundation in Java for customizing and automating networks | 3 | Topology, MITM |
| ONOS | Carrier-grade SDN controller based on Java | 3 | DDoS, Topology |
| Flowvisor | OpenFlow controller that serves as a network hypervisor. Its codebase is primarily in Java | 1 | DDoS |

**Table 9** Traditional datasets

| Dataset | Year | Features | Brief description |
|---|---|---|---|
| Darpa | 1999 | 41 | First intrusion detection evaluation dataset, simulated military network traffic |
| KDD-Cup | 1999 | 41 | Derived from DARPA, used for network intrusion detector learning |
| NSL-KDD | 2009 | 41 | Refined version of KDD-Cup, addressing some of its issues |
| UNB ISCX | 2012 | 20 | Realistic network traffic with labeled normal and attack activities |
| CIC-IDS 2018 | 2018 | 80 | Modern dataset with diverse attack scenarios and up-to-date normal traffic |

### 7.1 Specific SDN Datasets

Over the past five years, numerous researchers have proposed new datasets generated in an SDN environment. However, during the review conducted for this survey, only two of these datasets were able to be accessed [50, 56]. Some studies failed to provide access to their repository, while others were simply unreachable.

The study conducted by [57] offers an intriguing perspective on detecting malicious controllers, indirectly addressingthe first research question posed in this survey. In this case, data is collected by extracting statistics from data plane devices that operate independently of potentially compromised controllers. The features considered include the switch participation index, the average fraction of switches per flow, the average degree of nodes, the priority frequency spike index, the timeout frequency spike index, the variance of drop actions, the number of switches, and the packet-in to packet-out ratio. Data collection in the study is conducted at the switches and then processed offline to derive the features above. This extracted information is subsequently submitted to a machine-learning module for both training and classification. The authors have modified the Ryu controller's code to simulate both legitimate and compromised controllers, with the relevant Python files made available to facilitate replication of their work. Nevertheless, the authors do not publish the actual dataset itself.

Additionally, the authors of [58] constructed a virtual SDN test network composed of three sections: Mininet, which includes a vulnerable web server (DVWA); a server farm; and a host acting as an attacker in Kali Linux. These components are connected to an Open Virtual Switch (OVS) controlled by an ONOS server. Seven types of attacks (DoS, DDoS, Web attacks, R2L, Malware, Probe, and U2R) are launched against the servers, and the classes of the dataset match those, plus the benign category. Data collection is achieved through processing pcap files with the CICFlowmeter tool [59], which provides 48 features. The dataset contains 343,939 instances, of which 68,424 represent normal traffic, and the remaining 275,515 constitute attack instances. Although the authors indicated that their dataset is available, neither of the provided links was functional at the time of writing this survey.

In another study, the authors of [60] propose a dataset where data collection is performed using flows and port statistics, yielding a total of 23 features. The authors detail only seven of these features: Average Packet Count per Flow, Average Byte Count per Flow, Protocol, Duration, Number of packet-in Messages, Packet Rate, and Port Bandwidth. The dataset consists of 1,04,345 records and utilizes the Ryu controller. Labeling of the dataset is performed automatically via a controller application, with just two categories: malicious (1) and benign (0). However, this dataset is not publicly accessible.

Nevertheless, in [50] a dataset designed to simulate ARP attacks in an SDN environment that results in a man-in-the-middle (MITM) attack is presented. The authors collect data from the packet-in handler method of the Ryu controller and match the Ethernet type field to compile 17 features. These include Source MAC Address at Ethernet, Source MAC Address at ARP Header, Sender IP Address in ARP Request, Target IP Address in ARP Request, Protocol Code, in-port, out-port, Destination MAC Address at ARP, Destination MAC Address at Ethernet, Time to Live (TTL), Switch-ID, Source Port, Ping Statistics, Destination Port, Operation Code, Round Trip Time, Packet Loss, and Number of packet-in Messages. The dataset comprises a total of 1,340,000 records. It adopts binary classifications, with 1 representing malicious traffic and 0 indicating benign traffic.

A different approach is presented in [56], which involves the creation of a dataset specifically designed for IoT. The testbed configuration comprises a single switch controlled by an ONOS server, serving both benign and malicious hosts in addition to IoT devices. Data collection is facilitated by an SDN application that retrieves flow entries from the network switch every second, generating 33 distinct features. The dataset categorizes the traffic into six classes: Normal, DoS, DDoS, Port scanning, OS with service detection, and Fuzzing. The total size of the dataset is 27 MB.

Authors like those of [61, 62] also present results obtained from their proprietary datasets, which consider attacks on network resources instead of the controller or the SDN architecture. Regrettably, these datasets are not publicly available.

The aforementioned studies propose methodologies for collecting data to create datasets for detecting attacks within an SDN network. However, they do not outline a specific set of attacks that target the controller, nor do they consider threats such as topology poisoning.

Table 10 presents a list of SDN-specific datasets created for a diverse type of attack. Key features used for the studies are listed below:

- Anand et al. [57] focuses on switch and network-wide statistics such as switch participation index and the average fraction of switches per flow. The features presented are sophisticated (i.e., frequency spike index), requiring a more complex analysis of switch behavior and flow patterns.
- Elsayed et al. [58] considers packet-based attributes and time-related ones. This approach provides a more comprehensive set with different dimensions of the network.
- Ahuja et al. [60] is more specific in the flow aspects of the network, with elements related to OpenFlow operation.
- Ahuja et al. [50] includes protocol-specific information since the study seeks to detect ARP (Address Resolution Protocol) attacks.

The granularity of the features also varies across the studies, with aggregated statistics and micro-level elements such as packet details. Balancing both types will significantly affect the computational efficiency of the collection.

### 7.2 Datasets Pertinence

Datasets are integral to research, providing a basis for modeling, analysis, and hypothesis testing. The requirements for suitable datasets have changed with the emergence of next-generation networks such as 5G and IoT. However, current research is still conducted using datasets that are over 20 years old and in networks that need to accurately reflect the complexity and heterogeneity of present and future networks. These studies overlook the

**Table 10** SDN-specific datasets

| Ref | Samples | Features | Target | Attack |
|---|---|---|---|---|
| [57] | Undetermined | 9 | OpenFlow packet traces from SDN data plane | Packet-in injection, Flow-rule modification, replication of packets |
| [58] | 343,939 | 48 | Transport layer session information from CIC Flowmeter | Seven attacks to network resources plus five SDN specific attacks |
| [60] | 104,345 | 23 | Collection of various flow and port statistics at a regular monitoring interval | DDoS Attacks with Packet-in injection |
| [50] | 1,340,000 | 17 | Examination of Packet-in messages | ARP-based MITM attack |

consideration of current protocols, architectures, usage patterns, and rapid and non-deterministic changes in network operations.

A suitable dataset should comprehensively document the various types of attacks that target the SDN controller. This includes the attacks' characteristics, their manifestations in the network, and their impact on the controller's performance and functionality. The dataset should also provide insight into the effective strategies for detecting these attacks.

Regarding the features, researchers should consider different SDN-specific features, as presented in the previous section. On [50, 60], authors used flow-level features and considered L2 elements such as MAC addresses. On the other hand, Elsayed et al. [58] used the CICFlowmeter tool, which gathers information from L4 flows. The most diverse features are those presented on [57], which focuses on switch and network-wide statistics such as switch participation index and the average fraction of switches per flow. It is crucial to consider the complexity of collecting the features and their representativeness for the dataset to allow replicability and future real-time applications. It is also essential to balance aggregated statistics and micro-level elements such as packet details.

Normal operations and under-attack scenarios should be included. The dataset should be current, reflecting the most recent trends and patterns in network behavior and attacks. Lastly, for broader applicability and comparative analysis, it should comply with standard formats and be publicly accessible. Therefore, creating datasets that capture this diversity and complexity is a challenging yet necessary endeavor for future research.

## 8 Discussion and Prospective

The vulnerabilities most frequently highlighted are the need to send packet-in messages to the controllers when there is a table-miss in the data plane and the exchange of LLDP messages to discover the topology, both intrinsic components of the OpenFlow protocol. Despite numerous countermeasures proposed by authors, research into alternative methods for handling table misses and discovering topology, or ways to secure these processes as part of the standard, is an essential next step.

Although the terms "saturation" or "disruption" of the control channel are frequently used in the literature, the fundamental method employed is the flooding of packet-in messages to sever communication between the controller and the data plane. This approach essentially categorizes bandwidth attacks under the same umbrella as DDoS attacks.

While Floodlight and Ryu are the most popular controllers, they may not provide the up-to-date elements needed for the network requirements in the coming years. The reason for this situation could be their ease of use, particularly for adapting new functions. The convenience of collecting data on both platforms might be the primary driver for researchers to use these controllers. Researchers should start exploring with more current controllers such as ODL and ONOS, which, according to our review, are the more advanced options. The challenge lies in ensuring that the steep learning curve associated with these advanced controllers does not deter researchers. Moreover, various attacks and mitigation mechanisms across different controller platforms must be tested to establish their performance and behavior.

The detection methods most utilized are those typically used in intrusion detection systems: statistical and machine learning algorithms. However, a few other methods for detecting the attacks have been proposed, particularly in the case of topology poisoning. This type of attack can be detected by probing links and maintaining data to assess the

consistency of the new topology information. Looking forward, the task is to refine these detection methods and ensure they are effective in identifying the full range of possible attacks, while also minimizing false positives and negatives.

Considering the dataset for that line of work, this survey found that several papers use benchmark datasets to test their SDN method. However, the dynamics of the network are different. Although some authors have created public datasets based on SDN, none have considered attacks directed to the SDN infrastructure. Unfortunately, in most cases, authors have not made their datasets public. Future work needs to focus on creating comprehensive, publicly available datasets that reflect the realities of SDN attacks. This will enable more robust testing and validation of proposed detection methods and facilitate the development of more effective solutions.

## 9 Future Research Directions

Considering the findings, researchers have five directions to follow, as presented below.

A critical area is improving OpenFlow protocol security measurements, particularly because they depend on unverified packet-in messages and LLDP topology discoveries. Both mechanisms present severe vulnerabilities. To tackle these issues, researchers could devise methods to validate messages or propose secure encapsulation.

Another concern is the broad spectrum of attacks in current networks, in contrast to the few categories for attack detection in the surveyed studies. The grouping of similar attacks presents the classification algorithms with difficulties in distinguishing different or new attacks. Detection frameworks should use algorithms that can classify extensive types of attacks.

Regarding the controllers, the main barrier to use newer, more sophisticated tools relies on the complexity of the operations. Specially for research purposes such as collecting data or extensibility of the functions. The future work should consider frameworks that acilitate research with advanced controllers, and define cross-controller communications and testing methodologies.

Attack detection methods are also a topic that requires attention from researchers. Although both statistical and machine learning, out of the surveyed proposals, only two of them considered more than one stage for detection, leaving a high risk of getting false positive/negative events. This suggests that it is necessary to create hybrid detection methods that help improve performance. Real-time capabilities also require attention since they are a large blind spot for the community.

Finally, the availability of relevant datasets is critical for the development of the field. The lack of public and specific SDN datasets is an obstacle to working on any of the topics mentioned previously. It is also necessary to find methodologies that validate those datasets, as well as specific metrics for evaluation.

## 10 Conclusion

In this survey, four critical research questions were addressed concerning the threats targeting the SDN controller, detection methodologies, the controllers deployed in various studies, and the datasets utilized for machine learning detection methods.

The most important finding was identifying SDN attacks targeting the controller and OpenFlow protocol, as well as recent machine learning detection methods and mitigation techniques.

The investigation revealed that most attacks primarily exploit the vulnerabilities inherent in OpenFlow, with a few exceptions, such as ICMP vulnerabilities. As for the detection methodologies, statistical and machine learning approaches remain predominant, supplemented by some methods that compare against previously collected data about network topology.

Examining the controllers utilized in the studies disclosed that Floodlight and Ryu are the most frequently employed. However, these appear to be outdated projects, while contemporary, actively supported controllers such as ONOS and ODL have received minimal attention in the research community.

Regarding suitable datasets for training machine learning models, the survey indicates that only two datasets tailored for SDN environments are publicly available. Both datasets need to consider attacks directed at the controllers, illustrating a gap in the current body of research.

**Funding** Open Access funding provided thanks to the CRUE-CSIC agreement with Springer Nature. Not applicable.

**Availability of Data and Materials** Not applicable.

## Declarations

**Conflict of interest** The authors declare that there are no Conflict of interest.

**Ethical Approval** Not applicable.



## References

1. Kreutz, D., Ramos, F. M., Verissimo, P. E., Rothenberg, C. E., Azodolmolky, S., & Uhlig, S. (2014). Software-defined networking: A comprehensive survey. *Proceedings of the IEEE, 103*(1), 14–76.
2. McKeown, N., Anderson, T., Balakrishnan, H., Parulkar, G., Peterson, L., Rexford, J., Shenker, S., & Turner, J. (2008). Openflow: Enabling innovation in campus networks. *ACM SIGCOMM Computer Communication Review, 38*(2), 69–74.
3. The P4 Language Consortium: P4 16 Language Specification. Accessed July 18, 2023, fromhttps://p4.org/p4-spec/docs/p4-16-working-draft.html
4. Barakabitze, A. A., Ahmad, A., Mijumbi, R., & Hines, A. (2020). 5g network slicing using sdn and nfv: A survey of taxonomy, architectures and future challenges. *Computer Networks, 167*, 106984.
5. Benabbou, J., Elbaamrani, K., & Idboufker, N. (2019). Security in openflow-based sdn, opportunities and challenges. *Photonic Network Communications, 37*, 1–23.
6. Maleh, Y., Qasmaoui, Y., El Gholami, K., Sadqi, Y., & Mounir, S. (2023). A comprehensive survey on sdn security: Threats, mitigations, and future directions. *Journal of Reliable Intelligent Environments, 9*(2), 201–239.

7. Eliyan, L. F., & Di Pietro, R. (2021). Dos and ddos attacks in software defined networks: A survey of existing solutions and research challenges. *Future Generation Computer Systems, 122*, 149–171.
8. Alhijawi, B., Almajali, S., Elgala, H., Salameh, H. B., & Ayyash, M. (2022). A survey on dos/ddos mitigation techniques in sdns: Classification, comparison, solutions, testing tools and datasets. *Computers and Electrical Engineering, 99*, 107706.
9. Bhuiyan, Z. A., Islam, S., Islam, M. M., Ullah, A. A., Naz, F., & Rahman, M. S. (2023). On the (in) security of the control plane of sdn architecture: A survey. *IEEE Access, 11*, 91550–91582.
10. Rahouti, M., Xiong, K., Xin, Y., Jagatheesaperumal, S. K., Ayyash, M., & Shaheed, M. (2022). Sdn security review: Threat taxonomy, implications, and open challenges. *IEEE Access, 10*, 45820–45854.
11. Singh, J., & Behal, S. (2020). Detection and mitigation of ddos attacks in sdn: A comprehensive review, research challenges and future directions. *Computer Science Review, 37*, 100279.
12. Han, T., Jan, S. R. U., Tan, Z., Usman, M., Jan, M. A., Khan, R., & Xu, Y. (2020). A comprehensive survey of security threats and their mitigation techniques for next-generation sdn controllers. *Concurrency and Computation: Practice and Experience, 32*(16), 5300.
13. Brooks, M., & Yang, B. (2015). A man-in-the-middle attack against opendaylight sdn controller. In *Proceedings of the 4th annual ACM conference on research in information technology* (pp. 45–49).
14. Shan-Shan, J., & Ya-Bin, X. (2018). The apt detection method based on attack tree for sdn. In *Proceedings of the 2nd international conference on cryptography, security and privacy* (pp. 116–121).
15. Ravi, N., & Shalinie, S. M. (2021). Blacknurse-sc: A novel attack on sdn controller. *IEEE Communications Letters, 25*(7), 2146–2150. https://doi.org/10.1109/LCOMM.2021.3075898
16. Chen, K.-Y., Liu, S., Xu, Y., Siddhrau, I. K., Zhou, S., Guo, Z., & Chao, H. J. (2021). Sdnshield: nfv-based defense framework against ddos attacks on sdn control plane. *IEEE/ACM Transactions on Networking, 30*(1), 1–17.
17. Cherian, M., & Varma, S. L. (2023). Secure sdn-iot framework for ddos attack detection using deep learning and counter based approach. *Journal of Network and Systems Management, 31*(3), 54.
18. Singh, V., Rajarajeswari, S., Kanavalli, A., & Sanjeetha, R. (2022). Mitigation of ddos attack in sdn using table miss-entry. In *2022 4th international conference on circuits, control, communication and computing (I4C)* (pp. 6–11). IEEE.
19. Ravi, N., Shalinie, S. M., Lal, C., & Conti, M. (2020). Aegis: Detection and mitigation of tcp syn flood on sdn controller. *IEEE Transactions on Network and Service Management, 18*(1), 745–759.
20. Alshra'a, A. S., & Seitz, J. (2019). Using inspector device to stop packet injection attack in sdn. *IEEE Communications Letters, 23*(7), 1174–1177.
21. Imran, M., Durad, M. H., Khan, F. A., & Derhab, A. (2019). Reducing the effects of dos attacks in software defined networks using parallel flow installation. *Human-Centric Computing and Information Sciences, 9*, 1–19.
22. Mehr, S. Y., & Ramamurthy, B. (2019). An svm based ddos attack detection method for ryu sdn controller. In *Proceedings of the 15th international conference on emerging networking experiments and technologies* (pp. 72–73).
23. Goksel, N., & Demirci, M. (2019). Dos attack detection using packet statistics in sdn. In *2019 international symposium on networks, computers and communications* (ISNCC) (pp. 1–6). IEEE.
24. Agrawal, N., & Tapaswi, S. (2021). An sdn-assisted defense mechanism for the shrew ddos attack in a cloud computing environment. *Journal of Network and Systems Management, 29*(2), 12.
25. Ahmad, S., & Mir, A. H. (2023). Protection of centralized sdn control plane from high-rate packet-in messages. *International Journal of Information Security, 22*, 1–10.
26. Khamaiseh, S., Serra, E., Li, Z., & Xu, D. (2019). Detecting saturation attacks in sdn via machine learning. In *2019 4th international conference on computing, communications and security (ICCCS)* (pp. 1–8). IEEE.
27. Cui, J., Zhang, J., He, J., Zhong, H., & Lu, Y. (2020). Ddos detection and defense mechanism for sdn controllers with k-means. In *2020 IEEE/ACM 13th international conference on utility and cloud computing (UCC)* (pp. 394–401). IEEE.
28. Li, J., Qin, S., Tu, T., Zhang, H., & Li, Y. (2022). Packet injection exploiting attack and mitigation in software-defined networks. *Applied Sciences, 12*(3), 1103.
29. Zhan, X., Chen, M., Yu, S., & Zhang, Y. (2019). Adaptive detection method for packet-in message injection attack in sdn. In *Algorithms and architectures for parallel processing: 19th international conference, ICA3PP 2019, Melbourne, VIC, Australia, December 9–11, Proceedings, Part II 19* (pp. 482–495) (2020). Springer.
30. Gao, D., Liu, Z., Liu, Y., Foh, C. H., Zhi, T., & Chao, H.-C. (2018). Defending against packet-in messages flooding attack under sdn context. *Soft Computing, 22*, 6797–6809.

31. Khellah, F. (2019). Control plane packet-in arrival rate analysis for denial-of-service saturation attacks detection and mitigation in software-defined networks. *Arabian Journal for Science and Engineering, 44*(11), 9349–9362.
32. Perez-Diaz, J. A., Valdovinos, I. A., Choo, K.-K.R., & Zhu, D. (2020). A flexible sdn-based architecture for identifying and mitigating low-rate ddos attacks using machine learning. *IEEE Access, 8*, 155859–155872.
33. Aldabbas, H., & Amin, R. (2021). A novel mechanism to handle address spoofing attacks in sdn based iot. *Cluster Computing, 24*(4), 3011–3026.
34. Soltani, S., Shojafar, M., Mostafaei, H., Pooranian, Z., & Tafazolli, R. (2021). Link latency attack in software-defined networks. In *2021 17th international conference on network and service management (CNSM)* (pp. 187–193). IEEE.
35. Gao, Y., & Xu, M. (2022). Defense against software-defined network topology poisoning attacks. *Tsinghua Science and Technology, 28*(1), 39–46.
36. Shrivastava, P., & Kataoka, K. (2021). Topology poisoning attacks and prevention in hybrid software-defined networks. *IEEE Transactions on Network and Service Management, 19*(1), 510–523.
37. Bui, T., Antikainen, M., & Aura, T. (2019). Analysis of topology poisoning attacks in software-defined networking. In *Secure IT systems: 24th Nordic conference, NordSec 2019, Aalborg, Denmark, November 18–20, 2019, Proceedings 24* (pp. 87–102). Springer.
38. Kaur, N., Singh, A. K., Kumar, N., & Srivastava, S. (2017). Performance impact of topology poisoning attack in sdn and its countermeasure. In *Proceedings of the 10th international conference on security of information and networks* (pp. 179–184).
39. Kong, D., Shen, Y., Chen, X., Cheng, Q., Liu, H., Zhang, D., Liu, X., Chen, S., & Wu, C. (2022). Combination attacks and defenses on sdn topology discovery. *IEEE/ACM Transactions on Networking, 31*(2), 904–919.
40. Macwan, S., & Lung, C.-H. (2019). Investigation of moving target defense technique to prevent poisoning attacks in sdn. In *2019 IEEE World Congress on Services (SERVICES)* (Vol. 2642, pp. 178–183). IEEE.
41. Smyth, D., Scott-Hayward, S., Cionca, V., McSweeney, S., & O'Shea, D. (2023). Secap switch-defeating topology poisoning attacks using p4 data planes. *Journal of Network and Systems Management, 31*(1), 28.
42. Alimohammadifar, A., Majumdar, S., Madi, T., Jarraya, Y., Pourzandi, M., Wang, L., & Debbabi, M. (2018). Stealthy probing-based verification (spv): An active approach to defending software defined networks against topology poisoning attacks. In *Computer Security: 23rd European symposium on research in computer security, ESORICS 2018, Barcelona, Spain, September 3–7, 2018, Proceedings, Part II 23* (pp. 463–484). Springer.
43. Huang, X.-B., Xue, K.-P., Xing, Y.-T., Hu, D.-W., Li, R., & Sun, Q.-B. (2022). An efficient scheme to defend data-to-control-plane saturation attacks in software-defined networking. *Journal of Computer Science and Technology, 37*(4), 839–851.
44. Li, Z., Xing, W., Khamaiseh, S., & Xu, D. (2019). Detecting saturation attacks based on self-similarity of openflow traffic. *IEEE Transactions on Network and Service Management, 17*(1), 607–621.
45. Xie, R., Cao, J., Li, Q., Sun, K., Gu, G., Xu, M., & Yang, Y. (2022). Disrupting the sdn control channel via shared links: Attacks and countermeasures. *IEEE/ACM Transactions on Networking, 30*(5), 2158–2172.
46. Rasool, R. U., Ashraf, U., Ahmed, K., Wang, H., Rafique, W., & Anwar, Z. (2019). Cyberpulse: A machine learning based link flooding attack mitigation system for software defined networks. *IEEE Access, 7*, 34885–34899. https://doi.org/10.1109/ACCESS.2019.2904236
47. Dixit, V. H., Doupé, A., Shoshitaishvili, Y., Zhao, Z., & Ahn, G.-J. (2018). Aim-sdn: Attacking information mismanagement in sdn-datastores. In *Proceedings of the 2018 ACM SIGSAC conference on computer and communications security* (pp. 664–676).
48. Alshamrani, A., Myneni, S., Chowdhary, A., & Huang, D. (2019). A survey on advanced persistent threats: Techniques, solutions, challenges, and research opportunities. *IEEE Communications Surveys & Tutorials, 21*(2), 1851–1877.
49. Zhou, H., Zheng, Y., Jia, X., & Shu, J. (2023). Collaborative prediction and detection of ddos attacks in edge computing: A deep learning-based approach with distributed sdn. *Computer Networks, 225*, 109642.
50. Ahuja, N., Singal, G., Mukhopadhyay, D., & Nehra, A. (2022). Ascertain the efficient machine learning approach to detect different arp attacks. *Computers and Electrical Engineering, 99*, 107757.

51. Tchendji, V. K., Mvah, F., Djamegni, C. T., & Yankam, Y. F. (2021). E2basep: Efficient bayes based security protocol against arp spoofing attacks in sdn architectures. *Journal of Hardware and Systems Security, 5*, 58–74.
52. Xu, Y., Ma, J., & Zhong, S. (2020). Detection and defense against ddos attack on sdn controller based on spatiotemporal feature. In *Security and privacy in digital economy: First international conference, SPDE 2020, Quzhou, China, October 30–November 1, 2020, Proceedings 1* (pp. 3–18). Springer.
53. Rangisetti, A. K., Dwivedi, R., & Singh, P. (2021). Denial of arp spoofing in sdn and nfv enabled cloud-fog-edge platforms. *Cluster Computing, 24*(4), 3147–3172.
54. Zhu, L., Karim, M. M., Sharif, K., Xu, C., Li, F., Du, X., & Guizani, M. (2020). Sdn controllers: A comprehensive analysis and performance evaluation study. *ACM Computing Surveys (CSUR), 53*(6), 1–40.
55. Arevalo Herrera, J., & Camargo, J. E. (2019). A survey on machine learning applications for software defined network security. In *Applied Cryptography and Network Security Workshops: ACNS 2019 Satellite Workshops, SiMLA, Cloud S &P, AIBlock, and AIoTS, Bogota, Colombia, June 5–7, 2019, Proceedings 17* (pp. 70–93). Springer.
56. Sarica, A. K., Angin, P. (2020). A novel sdn dataset for intrusion detection in iot networks. In *2020 16th international conference on network and service management (CNSM)* (pp. 1–5). IEEE.
57. Anand, N., Babu, S., & Manoj, B. (2018). On detecting compromised controller in software defined networks. *Computer Networks, 137*, 107–118.
58. Elsayed, M. S., Le-Khac, N.-A., & Jurcut, A. D. (2020). Insdn: A novel sdn intrusion dataset. *IEEE Access, 8*, 165263–165284.
59. Draper-Gil, G., Lashkari, A. H., Mamun, M. S. I., & Ghorbani, A. A. (2016). Characterization of encrypted and vpn traffic using time-related. In *Proceedings of the 2nd international conference on information systems security and privacy (ICISSP)* (pp. 407–414).
60. Ahuja, N., Singal, G., Mukhopadhyay, D., & Kumar, N. (2021). Automated ddos attack detection in software defined networking. *Journal of Network and Computer Applications, 187*, 103108.
61. Huang, C.-H., Lee, T.-H., Chang, L.-h., Lin, J.-R., & Horng, G. (2019). Adversarial attacks on sdn-based deep learning ids system. In *Mobile and wireless technology 2018: International conference on mobile and wireless technology (ICMWT 2018)* (pp. 181–191). Springer.
62. Santos, R., Souza, D., Santo, W., Ribeiro, A., & Moreno, E. (2020). Machine learning algorithms to detect ddos attacks in sdn. *Concurrency and Computation: Practice and Experience, 32*(16), 5402.




**Juliana Arevalo-Herrera** is a Telecommunications Engineer from Universidad Santo Tomás and an MSc in ICT Security from Universitat Oberta de Catalunya. She works as an assistant professor at Universidad Santo Tomás, Bogotá, and is pursuing a Ph.D at Universidad Rey Juan Carlos, Spain.

**DR. Jorge Camargo Mendoza** is Associate Professor at the Department of Computing Systems and Industrial Engineering at the Universidad Nacional de Colombia, where he leads the UnSecure Lab research group focused on Cybersecurity. He earned a Computing Systems Engineer degree at Universidad Nacional de Colombia, a MSc. in Computer Systems Engineering at Universidad de los Andes, and a Ph.D. in Computer Systems Engineering at Universidad Nacional de Colombia.

**Jose Ignacio Martínez Torre** received the degree in physics from Cantabria University, Santander, Spain, in 1991, and the Ph.D. degree in physics from the Complutense University of Madrid, Madrid, Spain, in 1999. Since then, he has been researching and teaching with the Rey Juan Carlos University of Madrid, Madrid, where he leads the Hardware Software Design Group. He has been the Head of the Computer Engineering Department, Computer Engineering School of the URJC for five years. He is author of more than 40 research papers and has been working in more than ten research and development projects in collaboration with universities and companies. He has been the supervisor of four Ph.D. Thesis and co-author of two patents in the area of electronic implementations related to the reconfiguration and Big Data computation.

**Tatiana Zona-Ortiz** was born in 1978 in Bogotá, Colombia. She is a Telecommunication Engineer (2002) from the Universidad Santo Tomás, Colombia. She holds a Ph.D in Telecommunications (2008) and a M.Sc degree in Project Management (2006) both from UPV, Spain. She was a researcher in ITACA Institute at UPV, from 2002 to 2007. Then she was researcher and R+D manager at ITACA-CT with a TORRES QUEVEDO grant until 2012. Currently, she is Full Professor at Universidad Santo Tomás in Facultad TIC. Her research interests also include R+D management, Smart cities, IoT, microwave heating, Electromagnetic fields Exposure, microwaves and propagation.

**Juan M. Ramírez** received a B.S. in electrical engineering, an M.S. in biomedical engineering, and a Doctor degree in applied sciences from the Universidad de Los Andes, Venezuela, in 2002, 2007, and 2017, respectively. From 2004 to 2019, he was an Associate Professor at the Electrical Engineering Department at Universidad de Los Andes, Venezuela. In addition, He spent a postdoctoral internship at the High Dimensional Signal Processing (HDSP) group at the Universidad Industrial de Santander, Colombia. He also was a Got Energy Talent GET-COFUND Marie Skłodowska-Curie Actions fellow at the Department of Computer Science from Universidad Rey Juan Carlos, Spain, from 2019 to 2021. He is currently a postdoctoral researcher at the IMDEA Networks Institute in Spain.

## Authors and Affiliations

**Juliana Arevalo-Herrera[1,3] · Jorge Camargo Mendoza[2] · Jose Ignacio Martínez Torre[1] · Tatiana Zona-Ortiz[3] · Juan M. Ramirez[4]**

✉ Juliana Arevalo-Herrera
j.arevaloh.2019@alumnos.urjc.es

Jorge Camargo Mendoza
jecamargom@unal.edu.co

Jose Ignacio Martínez Torre
joseignacio.martinez@urjc.es

Tatiana Zona-Ortiz
angelazona@usta.edu.co

Juan M. Ramirez
juan.ramirez@imdea.org

1 Universidad Rey Juan Carlos, Madrid, Spain

2 Universidad Nacional de Colombia, Bogotá, Colombia

3 Universidad Santo Tomás, Bogotá, Colombia

4 IMDEA Networks Institute, Madrid, Spain